

GRP-HEFT: A Budget-Constrained Resource Provisioning Scheme for Workflow Scheduling in IaaS Clouds

Hamid Reza Faragardi^{ID}, Mohammad Reza Saleh Sedghpour^{ID}, Saber Fazliahmadi, Thomas Fahringer^{ID}, *Member, IEEE*, and Nayereh Rasouli^{ID}

Abstract—In Infrastructure as a Service (IaaS) Clouds, users are charged to utilize cloud services according to a pay-per-use model. If users intend to run their workflow applications on cloud resources within a specific budget, they have to adjust their demands for cloud resources with respect to this budget. Although several scheduling approaches have introduced solutions to optimize the makespan of workflows on a set of heterogeneous IaaS cloud resources within a certain budget, the hourly-based cost model of some well-known cloud providers (e.g., Amazon EC2 Cloud) can easily lead to a higher makespan and some schedulers may not find any feasible solution. In this article, we propose a novel resource provisioning mechanism and a workflow scheduling algorithm, named Greedy Resource Provisioning and modified HEFT (GRP-HEFT), for minimizing the makespan of a given workflow subject to a budget constraint for the hourly-based cost model of modern IaaS clouds. As a resource provisioning mechanism, we propose a greedy algorithm which lists the instance types according to their efficiency rate. For our scheduler, we modified the HEFT algorithm to consider a budget limit. GRP-HEFT is compared against state-of-the-art workflow scheduling techniques, including MOACS (Multi-Objective Ant Colony System), PSO (Particle Swarm Optimization), and GA (Genetic Algorithm). The experimental results demonstrate that GRP-HEFT outperforms GA, PSO, and MOACS for several well-known scientific workflow applications for different problem sizes on average by 13.64, 19.77, and 11.69 percent, respectively. Also in terms of time complexity, GRP-HEFT outperforms GA, PSO and MOACS.

Index Terms—Cloud computing, workflow scheduling, resource provisioning, budget-constrained scheduling

1 INTRODUCTION

THE cloud provides a highly scalable pool of resources to execute real-world applications faster than ever which is of paramount importance for computation-intensive applications such as astronomy, earthquake analysis, and genetic science. Cloud computing enables to run applications on an almost infinite number of powerful servers which can substantially advance such applications and their corresponding domain sciences.

Workflow is a common model to describe scientific applications where a set of tasks creates the nodes of a workflow and the communication between tasks forms the links between nodes. Users aim at running their applications with the shortest possible execution time, implying to use the most powerful computing resources of the cloud. As IaaS cloud providers charge users for powerful resources more than for slower ones, higher computation speed proportionally causes higher

economic cost. For example, in Amazon EC2, the cost of using a very powerful instance is almost 15 times more expensive than the least powerful instance.¹ Accordingly, using the fastest computing resources can dramatically increase the execution cost of a given workflow. Thus cloud resources should be wisely chosen for the efficient execution of a given workflow application given a certain budget. Note that, hereon, we refer to a workflow application as a workflow for short, and we also refer to the economic cost of executing a workflow on cloud resources as the cost of a workflow.

Finding the fastest makespan of a workflow on heterogeneous cloud resources with a certain budget constraint is known as an NP-hard problem [1], [2]. Several studies already focused on minimizing makespan on a set of heterogeneous computing resources within a certain budget [3], [4]. Nevertheless, most of these studies cannot be applied for cloud environments. There are two major differences between cloud environments and traditional heterogeneous computing systems (e.g., Grid), namely, much greater resource pools, and complex pricing models.

- 1) Size of the resource pool: Cloud providers usually have multiple data centers with various instance types. For example, Amazon EC2 cloud offers more than 35 different instance types and eight regions. For any type a user can request any number of instances (initially

- H.R. Faragardi and T. Fahringer are with the Institute of Computer Science, University of Innsbruck, 6020 Innsbruck, Austria.
E-mail: {faragardi, tf}@dps.uibk.ac.at.
- M.R. Saleh Sedghpour and S. Fazliahmadi are with the Iran University of Science and Technology, Tehran 16844, Iran.
E-mail: m_saleh@alummi.iust.ac.ir, saberfazliahmadi@gmail.com.
- N. Rasouli is with the Department of Computer Engineering, Karaj Branch, Technical and Vocational University, Alborz 31975, Iran.
E-mail: nrasouli@yahoo.com.

Manuscript received 29 Mar. 2019; revised 10 Dec. 2019; accepted 16 Dec. 2019. Date of publication 20 Dec. 2019; date of current version 23 Jan. 2020.
(Corresponding author: Hamid Reza Faragardi.)
Recommended for acceptance by R. Yahyapour.
Digital Object Identifier no. 10.1109/TPDS.2019.2961098

1. These prices refer to generation of compute-optimized on-demand instances as of September 2018

100 instances but according to the negotiation and type of requested service, this number can increase to any arbitrary value), which results in a huge pool of instances, e.g., $100 \times 35 \times 8 = 28000$ instances. Section 3.1 further discusses how changing the cost model from the basic cost model to the hourly-based cost model can lead to a failure of classical budget-constrained algorithms. Consequently, when a more complex cost model (rather than the basic cost model) is applied, most of the previous studies are not promising.

Fortunately, there are a few studies focusing on budget constraint workflow scheduling for the hourly-based cost model such as [5], [6], [7], [8], [9]. However, they are not mature enough in terms of performance compared to numerous studies carried out in the context of the basic cost model scheduling algorithms. Therefore, our paper aims at improving the state-of-the-art for budget-constraint workflow scheduling in cloud environments where the objective function is to minimize the completion time of the given workflow (makespan) for the hourly-based cost model. Note that we use the term ‘makespan’ and ‘completion time of the workflow’ interchangeably.

The Proposed Method. GRP-HEFT contains two parts, (i) a resource provisioning algorithm and (ii) a scheduling algorithm. The resource provisioning algorithm specifies how many and which types of instances are taken from the infinite pool of resources offered by cloud providers, and the scheduler determines the assignment of tasks to the obtained instances and the execution order of tasks within each instance.

For evaluating the quality of the proposed solution framework, we compare GRP-HEFT with multiple state-of-the-art baseline algorithms. Furthermore, in order to present a comprehensive analysis of GRP-HEFT, we consider several realistic scientific workflows [10]. Experimental results demonstrate that GRP-Heft substantially outperforms state-of-the-art methods.

Contributions of the paper.

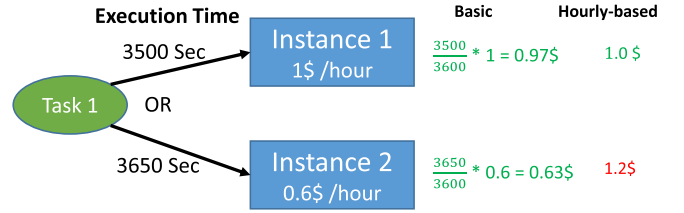
- 1) We propose a resource provisioning and a scheduling algorithm for efficient execution of workflows in cloud environments for an hourly-based cost model constrained by a budget.
- 2) As a resource provisioning mechanism, we propose a greedy algorithm which lists the instance types according to their efficiency ratio, and takes the most efficient instances constrained by the budget.
- 3) As the scheduler, we modify the best-known workflow scheduling algorithm on a set of heterogeneous resources, called Heterogeneous Earliest-Finish-Time (HEFT), such that it considers the given budget in the scheduling process.

Organization of the Paper. This paper is organized as follows. In Section 2, a comprehensive review of related work is presented. In Section 3, we describe the problem and assumptions, which is then followed by our proposed solution framework in Section 4. In Section 5, the performance results of GRP-HEFT are compared with other baselines. Finally, in Section 6, we conclude our paper by providing a summary along with future directions.

2 RELATED WORK

There are a number of best-known list-based heuristic algorithms for performance-effective workflow scheduling on a

Example 1: Budget = 1\$



Example 2: Budget = 1.2\$



Fig. 1. The behavior of classical scheduling algorithms for the hourly-based cost model.

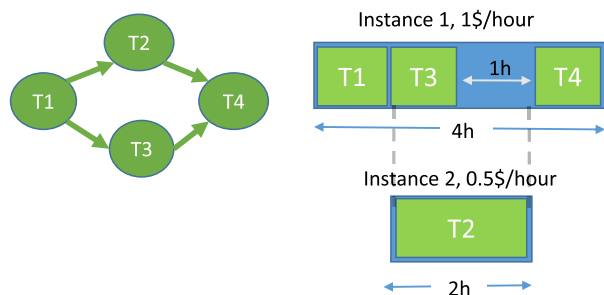
set of heterogeneous resources such as Heterogeneous Earliest Finish Time (HEFT) and Critical Path on a Processor (CPOP) [11], which are widely used in popular workflow management tools. A list-based heuristic first prioritizes the tasks according to a metric such as *Upward* and *Downward Ranking*, then assigns the tasks to the fastest processors. Although the traditional studies focus on minimizing only makespan, recent studies have begun to consider other important QoS requirements such as cost, security, and reliability [12]. Among all QoS constrained workflow scheduling problems in the IaaS cloud environment, makespan and monetary cost are two primary categories in the literature [13], [14].

The studies considering both makespan and cost for workflow scheduling in cloud environments can be divided into three major categories, A) The papers that optimize makespan while the cost is reflected as a budget constraint discussed in Section 2.1, B) The papers that optimize the cost of execution of workflows while the time is reflected as a deadline constraint are discussed in Section 2.2, and C) the papers that focus on both makespan and cost as a bi-objective optimization problem are discussed in Section 2.3.

2.1 Budget-Constrained Workflow Scheduling

This paper belongs to this category, therefore, we provide a more detailed discussion of this topic sub-divided into two sub-groups. 1. papers considering the basic cost model, and 2. studies considering the hourly-based cost model.

1. Basic Cost Model Studies. In [4], the authors proposed two budget-constrained algorithms called Loss and Gain to schedule DAG workflows in grid environments. The Loss algorithm starts from a given schedule proposed by a performance-effective scheduler such as HEFT and CPOP, and then they modify the cost of schedule by assigning the tasks to least-expensive resources until the budget is met. The Gain algorithm starts with a cost-effective schedule, then reschedule the workflow to improve the makespan until the cost of execution is close to the budget. Unfortunately, both Loss and Gain fail to deal with the hourly-based cost model since rescheduling a task from the most expensive instance to a least-expensive instance may even increase the cost of schedule (as discussed

Example 3: Budget = 4.0\$**Cost of Execution****Basic****Hourly-based**

$$3 * 1\$ + 2 * 0.5\$ = 4.0\$$$

$$4 * 1\$ + 2 * 0.5\$ = 5.0\$$$

Fig. 2. Ignoring the idle time intervals of the basic cost model.

in Fig. 1). In [5], the problem is formulated as an analytically model and it is proved that the problem is non-approximable. A heuristic algorithm called Critical-Greedy was introduced. Arabnejad *et al.* [3] proposed a Heterogeneous Budget Constrained Scheduling (HBCS) algorithm that guarantees an execution cost within the user's specified budget and minimizes the execution time of the user's application. Here again, HBCS is not suitable for the hourly-based cost model. In [15] a heuristic algorithm called Minimizing the Schedule Length using the Budget Level (MSLBL) was introduced. According to the reported results, MSLBL outperforms both HBCS and DBCS. DBCS [16] is a budget-deadline constrained algorithm which finds a feasible solution that satisfies both deadline and budget constraints. A heuristic algorithm, called Partial Critical Path Budget Balanced (PCP-B), was introduced in [14]. They employ a PCP-wise budget distribution mechanism to balance the budget among partial critical paths according to their sequential or parallel structure nature. The budget distribution mechanism is implemented using a binary search method. Moreover, a partially rescheduling mechanism is designed to further tune budget distribution.

Overall, the studies belonging to the basic cost model group cannot directly deal with the hourly-based budget constraint scheduling. The reasons were already discussed in Figs. 1 and 2.

2. *The Hourly-Based Cost Model Studies.* The work in [6] deals with budget-constrained workflow scheduling. An evolutionary algorithm based on Particle Swarm Optimization (PSO) is presented. PSO is the most recent work on budget-constrained workflow scheduling that considers the hourly-based cost model. Indeed, meta-heuristic algorithms such as PSO, Genetic Algorithm (GA), Ant Colony Optimization (ACO), etc. can deal with both basic and hourly-based cost model. Hence, PSO is one of the algorithms chosen to compare with our proposed method to evaluate its performance. However, as the considered problem is multi-dimensional (it includes both resource allocation and scheduling problems) and the problem space is extremely huge, classical meta-heuristic algorithms usually are not efficient to deal with the problem and more elaborated hybrid models (e.g., hybrid PSO [17]) and/or more intelligent problem-specific meta-heuristic operators (e.g., [18]) are needed.

2.2 Deadline-Constrained Workflow Scheduling

In [19], Rodriguez and Buyya focused on the static cost minimization deadline-constrained problem considering the elastic provisioning, heterogeneity of cloud resources, and VM performance variation. They use a particle swarm optimization algorithm for resource provisioning and scheduling. In [20] a genetic algorithm was proposed for deadline-constrained workflow scheduling where the co-evolution approach is used to adjust the crossover and mutation probability, which can accelerate the convergence and prevent prematurity. Deldari *et al.* proposed a heuristic algorithm that divides the workflow into multiple clusters. Thereafter, an extendable and flexible scoring approach chooses the best cluster combinations to achieve the goals of the algorithm [21]. Wu *et al.* in [22] applied a metaheuristic algorithm based on ant colony optimization as well as a simple heuristic, named ProLiS, to deal with deadline-constrained workflow scheduling. ProLiS distributes the deadline to each task, proportionally to a novel definition of probabilistic upward rank, and follows a two-step list scheduling methodology including rank tasks and sequentially allocates each task to a service which meets the sub-deadline and minimizes the cost. In [23] a Genetic Algorithm (GA) was applied to minimize the execution cost of scientific workflows under a certain deadline defined by the user. Although they use the hourly-based cost model, the different objective (minimizing execution cost rather than minimizing the makespan) and different constraints (deadline and security rather than the budget) of their work in comparison to what this paper addresses, places their work in another category (i.e., second category). Overall, this category of workflow scheduling problems is extremely popular. Most related work found in the literature on static workflow scheduling belongs to this category. In [12] and [24] a comprehensive survey and a holistic categorization of scientific workflow scheduling algorithms for IaaS clouds are provided.

2.3 Multi-Objective Workflow Scheduling

Multi-Objective Heterogeneous Earliest Finish Time (MOHEFT) [2] is a multi-objective optimization method which is an extension to the well-known HEFT. MOHEFT is a heuristic-based method that computes a set of tradeoff solutions with an additional overhead compared to HEFT. It builds several intermediate workflow schedules in parallel in each step instead of a single schedule. To ensure the quality of the tradeoff solutions, MOHEFT uses dominance relationships and a metric called crowding distance to guarantee their diversity. Garg *et al.* introduced a fuzzy-based particle swarm optimization approach to deal with the workflow scheduling problem for Grid architectures. A fuzzy dominance metric is used which quantifies the relative fitness of solutions for the multi-objective domain to generate the Pareto optimal solutions [25]. In [26], an evolutionary algorithm dubbed as EMS-C based on genetic algorithm is adopted where a new encoding scheme is proposed which represents all the scheduling orders, task-instance assignments, and instance specification choices. Based on this scheme, a set of new genetic operators, the evaluation function and the population initialization scheme are introduced

for the problem. Recently, Z.G. Chen *et al.* [27] introduced a Multi-Objective Ant Colony System (MOACS) based on a co-evolutionary multi-population scheme for the bi-objective (cost versus makespan) framework, which adopts two colonies to deal with these two objectives, respectively. They demonstrated that MOACS outperforms the other state-of-the-art methods such as EMS-C and MOHEFT.

2.4 Comparative Baselines

We used multiple comprehensive survey articles such as [12], [24], [28] that review and categorize workflow scheduling algorithms for scientific workflows. According to these surveys and also our extensive analysis of literature, we selected three major algorithms as baselines to compare against our new algorithm. Obviously, the main baseline is the latest budget-constrained workflow scheduling algorithm (i.e., Category A) based on PSO [6] that uses the same cost model as we use (i.e., the hourly based cost model). To have a holistic evaluation of the proposed method, we even go further and consider two algorithms from other categories (Category B and C) as the other baselines. Although other algorithms from different categories are not directly comparable with a budget-constrained scheduling algorithm, Section 5.1 explains how they can be compared with our GRP-HEFT algorithm. The metric to choose an algorithm from the list of papers of each category is *the latest published paper capable of considering the hourly based-cost model*. According to this metric, from Category B, GA [23] is opted as the second baseline, while from Category C, MOACS [27] is considered as the third baseline, respectively. The reasons to compare not only with single-objective methods but also with a multi-objective optimization method are 1) to provide a comprehensive performance analysis with several baseline algorithms, 2) to compare the performance of single-objective optimization methods with multi-objective optimization methods for the same problem class, 3) to include MOACS in our evaluation as the latest scheduling method that for most cases outperforms other previous methods such as MOHEFT, EMS/C, and PSO.

3 PROBLEM DESCRIPTION

The considered problem in this paper has three dimensions, namely, i) choosing a set of instances to run the given workflow under a certain budget, (ii) scheduling tasks to the set of chosen instances, (iii) controlling the execution order of tasks assigned to the same instance. It should be mentioned that when we refer to the optimal solution, we mean a solution achieved by an exhaustive search method considering all the possible combinations and select the best one found.

3.1 Basic Cost Model Versus Hourly-Based Cost Model

Using three examples, this subsection clarifies how changing the cost model from the basic cost model to the hourly-based cost model can impact the behavior of classical budget-constrained algorithms that use the basic cost model.

As shown in Fig. 1 for the first two examples, there is a single task that can be assigned either to instance 1 or 2. In the first example, the budget-constrained scheduling algorithm that uses the basic cost model which assigns Task 1 to the second instance increases the makespan but also raises the cost

and in our example even violates the budget. In the second example, assigning the task to the first instance leads to a budget violation. In the third example (Fig. 2), there is a workflow including four tasks which are scheduled on two instances. In this example, the execution of task 3 should be delayed until the data of the task 2 arrives. This results in a one-hour gap between task 1 and task 3 in the first instance. For the basic model, the execution cost is equal to the sum of the execution cost of each task irrespective of gaps between the tasks. Hence, the total cost of execution in the basic cost model is 4\$, whereas the cost for the hourly-based cost model implies a cost of 5\$ since the idle time of the first instance imposes additional execution costs for the workflow. In this example, this results in a budget violation, however, the algorithm using the basic cost model is unaware of this problem. These simple examples clearly answer the question *Why does this small change in the cost model lead to failing all the traditional budget-constrained workflow scheduling algorithms?*

3.2 Problem Modeling

A workflow is represented by a Directed Acyclic Graph (DAG), $G = (T, E)$, where T indicates a set of nodes, $T = \{t_1, t_2, \dots, t_N\}$, and each node is a task. Moreover, E represents a set of edges between tasks, which are control and/or data dependencies. The size of data transferred between t_i and t_j is denoted by $CR_{i,j}$.

A cloud resource pool is represented by a set of m different instance types $\{p_1, p_2, \dots, p_m\}$. The maximum number of instances which can be taken from each instance type is assumed to be equal to the number of tasks of the workflow. Since a task is not allowed to be split and run on more than one instance, this is a reasonable assumption that the number of required instances never exceeds the number of tasks. Hence, the total number of instances in the resource pool, denoted by M , is equal to $N \times m$ where N is the number of tasks in the given workflow. This is the same size of the resource pool that is used in [26] which will allow us to compare their method with list-based scheduling methods such as MOHEFT.

The processing power of the various instance types can vary. We use $e_{i,j}$ to indicate the execution time of the i th task on p_j . Instance type A is faster than instance type B , if p_A executes tasks in a shorter time than instances of p_B . The reason behind this speed up could be (i) a higher MIPS rate for the virtual processor of the instance type, (ii) a higher number of virtual cores, (iii) a larger memory size and storage capacity, and faster storage access time. Cloud providers specify the processing power of different types of the provided instances use a metric names Compute Unit (CU) (e.g., ECU used by Amazon EC2) or similar concepts such as *mean performance* [2]. CU_k denotes the compute unit of p_k . A higher value of CU potentially leads to higher compute power. The CU value does not only depend on the number of cores but also other parameters such as memory size. For example, in Amazon EC2, 'm1.medium' and 'm1.small' both include a single virtual CPU while the value of their CU is 3.75 and 1.7, respectively, which reflects the different memory size of those instance types.

Tasks Execution Times. For several scientific standard workflows, the execution time of tasks on Xeon@2.33 GHz

CPU (CU=8) has been published [10]. The execution time of tasks is reversely proportional to the CU value. This is the same notion commonly used to calculate the execution time of tasks of scientific workflows in the context of static workflow scheduling [2], [23], [26], [27]. We use the execution time of tasks for an instance with $CU = 1$ as the reference execution time of tasks and denoted by ref_time . Accordingly, the execution time of task t_i , running on an instance of j with the type of p_k , is

$$e_{i,j} = \frac{ref_time(t_i)}{CU_k}. \quad (1)$$

The price of each instance type p_k for one hour usage, denoted by $Price_k$, is given by the cloud provider which is an input to the problem description. The communication bandwidth of each instance type is known in advance, which can be found from the catalog of the instance types offered by the cloud provider. To calculate the communication time for sending data from p_k to p_q , we use the minimum bandwidth of the sender and the recipient instances. Note that, if both sender and recipient tasks are hosted by the same instance, we ignore the communication time.

Solution Representation. We need to adopt an efficient representation for each candidate solution of the considered problem. The solution representation specifies the data structures to be used by the algorithm and affects the algorithm performance [29]. Let us first introduce the problem space. The set of all possible combinations for (i) selecting a subset of cloud instances among the pool of instances, (ii) different assignment of tasks to the chosen subset of instances, and (iii) various order of execution of tasks, forms the problem space. Each point in the problem space corresponds to a possible assignment of the tasks of the workflow to a subset of cloud instances that potentially could be a solution for the problem. As the problem contains three sub-problems, we use three representations for each of these sub-problems, and the integration of these three representations indicates a complete candidate solution for the whole problem. We denote such a candidate solution with the assignment χ . Fig. 3 shows an illustrative example for the solution representation where a workflow is scheduled on three instances. An assignment χ includes the following elements:

- 1) For representing the assignment of tasks to instances, we use the vector X of N (the number of tasks) elements, and each element is an integer value between one and M (the maximum number of instances). When the i th element of the X array is j , means that the i th task is assigned to the j th instance. This representation fulfills the *no redundancy constraint*, meaning that each task should be assigned to no more than one instance.
- 2) To represent the type of each instance, we use the vector Y of M elements, and each element is an integer value between one and m (the number of instance types). When the j th element of the Y array is k , means that the type of the j th instance is k . It should be mentioned that in order to reduce complexity, the number of elements of Y can be reduced to the number of used instances, instead of M . This

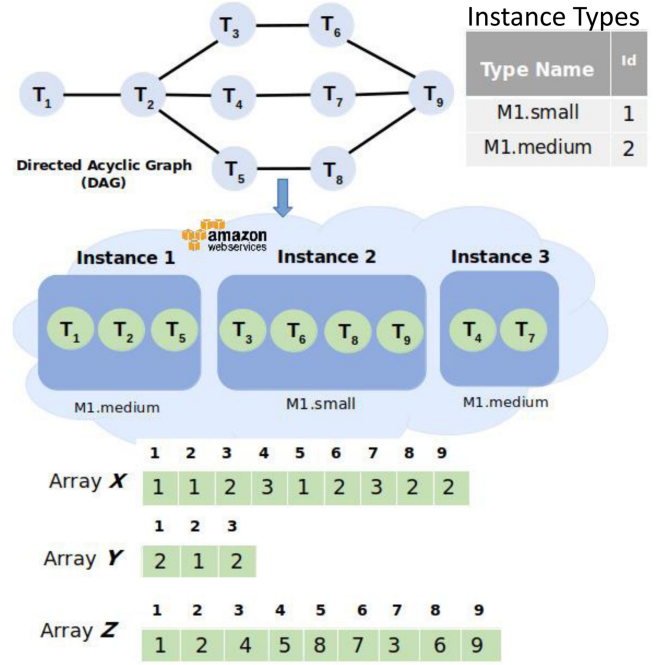


Fig. 3. Representation of a placement vector.

is shown in Fig. 3 where the length of Y is set to three since only three instances are used in the X array.

- 3) To represent the execution order of tasks, we use the vector Z of N elements, and each element is an integer value between one and N . In the Z array, when the position of task t_i is before the position of t_j , then t_i should be executed before task t_j . The point about the Z array is that the successors of a task should never appear before the task. In other words, the order of the elements of the Z array should comply with the topological sort of the DAG.

Makespan. The total elapsed time required to execute the entire workflow is called Makespan, which is calculated as follows:

$$StartTime_{t_i}(\chi) = \max \left\{ \max_{t_p \in pred_{t_i}} \{ FinishTime_{t_p}(\chi) + \frac{CR_{p,i}}{BW_{kj}} \}, CPU_Available_Time_j \right\}, \quad (2)$$

where k denotes the instance to which t_p is allocated, j indicates the instance to which t_i is allocated, and BW_{kj} denotes the bandwidth between instance k and j . Furthermore, $CPU_Available_Time_j$ denotes the CPU available time of the j th instance, which is the earliest time that the CPU(s) of the j th instance becomes available to execute t_i . As we use a non-preemptive scheduler, once a task starts to run on an instance, the CPU(s) is not reclaimed unless the execution of the task is completed. Accordingly, the finish time of a task is achieved by

$$FinishTime_{t_i}(\chi) = StartTime_{t_i}(\chi) + e_{i,j}(\chi). \quad (3)$$

The makespan of a workflow is the termination time of all tasks of the workflow, which implies

$$Makespan(\chi) = \max_{task_i \in T} \{ FinishTime_{t_i}(\chi) \}. \quad (4)$$

3.3 Optimization Problem Formulation

The optimization problem is formulated as follows:

$$\text{Minimize: } Maskespan(\chi), \quad (5a)$$

$$\text{Subject to: } TotalCost(\chi) \leq Budget, \quad (5b)$$

where $TotalCost(\chi)$ denotes the total cost of execution of the workflow according to the assignment χ , and calculated as the sum of computation cost and networking cost. In order to calculate the computation and communication costs, we need to know the cost model offered by the IaaS cloud provider.

Cost Model. As discussed before, most of the traditional studies focused on the basic cost model which is a popular cost model since grid computing was introduced. However, cloud providers usually use two other cost models including (i) charging the users according to instance-hours (hourly-based) where all partial hours are billed as full hours. For example, if a user runs an instance for 1.5 hours and the cost of this instance is 1\$, then the user is charged 2\$. This is the cost model used by Amazon EC2, and (ii) per-instance usage (also referred to as per-second cost model) where the actual usage time is used to calculate the bill. For example, if a user runs an instance for 1.5 hours and the cost of instance is 1\$, the cost is 1.5\$, nevertheless, in the per-instance usage cost model, similar to hourly-based cost model, the gap between the tasks running on the same instance is still counted in the cost model. Although there are some cloud providers offering other cost models such as the per-second cost model, we particularly focus on the hourly-based cost model, since it is widely used by large public cloud providers.

In our experiments, we use the hourly-based cost model provided by Amazon EC2. Amazon EC2 does not charge for internal data transfers among EC2 instances which do not require a public IP address. Hence, in the calculation of the total cost for running the workflow, the networking cost is considered zero. The total cost of an assignment is calculated by

$$TotalCost(\chi) = \sum_{\forall UsedInstances} Price(I_k) \times \left\lceil \frac{FT_{I_k}(\chi) - ST_{I_k}(\chi)}{3600} \right\rceil, \quad (6)$$

where the set $UsedInstances$ is a subset of all instances to which at least one task is assigned according to the assignment χ . FT_{I_k} and ST_{I_k} denotes the finish time and start time of the k th instance, respectively. $Price(I_k)$ is the cost of execution of the instance I_k for one hour usage, which is achieved according to the instance type and the price of the associated instance type. Since all the execution/communication times in our system are stated in second, we divide them by 3,600 to calculate the cost per hour.

It should be noted that the other cost models such as (i) the basic cost model which was extensively studied in the context of grid computing, and (ii) the per-instance usage cost model which is a common model for classical cloud scheduling algorithms are beyond the scope of this paper. If we aim to propose a generic algorithm suitable to work with all of these cost

models, we need to use meta-heuristic algorithms rather than problem-specific heuristics as proposed in our paper.

4 SOLUTION FRAMEWORK

As the problem is NP-hard, there is no polynomial algorithm to find the exact solution. In other words, the time complexity of the algorithms capable of finding the global optimal solution for this problem is exponential. Although we focus on an offline scheduling process executed before the actual execution of a workflow, an extremely long processing time is unacceptable. Therefore, an exhaustive search for finding a global optimum is unfeasible for even a small workflow with 100 tasks. Accordingly, a heuristic algorithm is needed to deal with such a complex NP-Hard problem to find near optimal solutions in a reasonable time.

Algorithm 1. Greedy Resource Provisioning

- 1: **Input:**(i) A DAG including a set of tasks, the execution time of the tasks, and size of data transferred between tasks, (ii) Budget to run the workflow, and (iii) a set of instance types.
 - 2: Assign all non-dominated instance types to the set InstTypeList
 - 3: Sort InstTypeList according to the efficiency rate of the instance types
 - 4: Best_assignment χ^{Best} = assignment χ = Null
 - 5: **repeat**
 - 6: Remaining_budget = Budget
 - 7: The set of taken instances $Y = \{\}$
 - 8: **while** (Remaining_budget $\geq$ Price($p^{Cheapest}$)) **do**
 - 9: Pick instance type p_i from the InstTypeList
 - 10: Take $n = \lfloor \frac{Remaining_budget}{Price_i} \rfloor$ instances from the type p_i and add them to Y
 - 11: Remaining_budget = Remaining_budget - Price $_i \times n$
 - 12: **end while**
 - 13: Run Modified-HEFT (Algorithm 2) with the given set of taken instances Y
 - 14: Calculate Makespan and TotalCost of the created assignment χ
 - 15: **if** (TotalCost(χ) $\leq$ Budget AND Makespan(χ) $\leq$ Makespan(χ^{Best})) **then**
 - 16: Update χ^{Best}
 - 17: **end if**
 - 18: Remove the first item from the InstTypeList
 - 19: **until** (InstTypeList = Null)
 - 20: **return** the assignment χ^{Best}
-

4.1 Proposed Method (GRP-HEFT)

The proposed method, Greedy Resource Provisioning + modified HEFT, has two major parts, comprising 1) resource provisioning which is responsible for choosing a subset of instances from all instances in the pool of cloud IaaS resources, and 2) resource scheduling which is in charge of assigning tasks to those instances that are provided by the resource provisioning.

4.1.1 Resource Provisioning Strategy

Our resource provisioning contains two phases to determine 1) which types of instances should be used, and 2) how many instances should be taken from each instance

type. The pseudo-code of the proposed resource provisioning algorithm is presented in Algorithm 1. For phase 1, we first check the list of instance types to remove the dominated instance types (line 2 of the algorithm). An instance type p_j is dominated by instance type p_i if $Price_i \leq Price_j$ and $CU_i \geq CU_j$. The remaining non-dominated instances are added to a list which is called *InstTypeList*. Afterwards, we sort the list in descending order according to the *efficiency rate* of the instance types (line 3). If there are multiple instance types with the same efficiency rate, the fastest one (i.e., higher CU) is prioritized. We define the efficiency rate as a metric to measure the ratio of the processing speed to the price of an instance, which is defined as follows:

$$EffRate_k = \frac{CU_k}{Price_k}. \quad (7)$$

It is worth mentioning that the reason to include only the CU unit and the price of the instance types in the calculation of the efficiency rate is due to the compute-intensive characteristic of scientific workflows [10]. Apparently, if we target data-intensive workflows such as stream-based applications, then the communication bandwidth of the instance types should be also considered in the calculation of the efficiency rate.

An instance with a higher efficiency rate (i.e., earlier position in the *InstTypeList*) is more desirable to be used for scheduling of the workflow. Hence, as the second phase where we should specify the number of taken instances from each instance type, we take as much as the budget allows from the most efficient instance type and add those instances to a list of taken instances (As mentioned in Section 3.2, this list of taken instances is kept by the Y array of the assignment χ of tasks to instances). The remaining budget (i.e., the budget that is not enough to buy any more most-efficient instance) is used to buy the second most efficient instance. This process continues until the remaining budget is not enough even for taking one more of the cheapest instances (lines 8-12). Note that $Price^{cheapest}$ denotes the price of the cheapest instance type.

The generated set of instances Y , is given as an input to a modified version of the well-known HEFT algorithm, and the scheduling phase begins (line 13). The modified-HEFT completes the assignment χ by filling the values of X and Z arrays. The details of modified-HEFT will be explained in Section 4.1.2. According to Eqs. (4) and (6), the makespan and the cost of execution of the workflow for the assignment χ are calculated (line 14). If the assignment χ is a feasible solution and it is better than the best solution found so far, then the best assignment is updated (lines 15-16). Thereafter, the most-efficient instance type is removed from *InstTypeList*, and the algorithm continues to the next iteration with the remaining items in the *InstTypeList*. The algorithm terminates when the list is empty. Eventually, the best assignment found across all iterations returns the final solution.

4.1.2 Resource Scheduling Strategy

The pseudocode of the modified-HEFT is presented in Algorithm 2. The modified-HEFT, similar to HEFT, consists of two phases, ranking and mapping. The main difference

between modified-HEFT and HEFT is that, once the execution time of an instance goes beyond full-hours, our modified-HEFT tries to avoid budget violations. Since the resource provisioning method does not consider the case where the run time of the provided instances goes beyond a full-hour, the budget constraint may be violated in the scheduling phase.

The ranking phase specifies the order in which the tasks are mapped to the available resources. To do so, the Upward-rank metric [11] is employed which implies the distance of each task to the end of the workflow. The idea of this ranking is to execute the tasks with more dependent tasks, earlier. Once specifying the execution order is done (line 2 of Algorithm 2), the second phase begins which assigns the tasks to the resources.

For each task t_i , we keep two parameters, 1) the minimum finish time of the task on all instances, and 2) the minimum finish time of the tasks on those instances that assigning t_i to them does not change the execution cost of those instances. Recall that, if adding the task to the instance does not increase the running time of the instance beyond a full-hour, then cost of running the instance remains unchanged. That is the reason to use I_{min}^* and $FinishTime_{min}^*$ in lines 4 and 5. I_{min}^* keeps the best instance for which adding the task does not change the instance cost.

Algorithm 2. Modified-HEFT

```

1: Input: (i) A DAG including the task set, associated execution times, and size of data transferred between tasks, and (ii) A set of taken instance set  $Y$ 
2: Sort all tasks of the DAG according to the  $Rank_u$ 
3: for each task  $t_i$  in DAG do
4:    $I_{min} = Null, I_{min}^* = Null$ 
5:    $FinishTime_{min} = \infty, FinishTime_{min}^* = \infty$ 
6:   for each resource  $I_j$  in  $Y$  do
7:     Calculate  $FinishTime_{i,j}$ 
8:     if ( $FinishTime_{i,j} < FinishTime_{min}$ ) then
9:        $FinishTime_{min} = FinishTime_{i,j}$ 
10:     $I_{min} = I_j$ 
11:   end if
12:   if (Adding_Not_Change_the_Cost( $t_i, I_j$ ) and ( $FinishTime_{i,j} < FinishTime_{min}^*$ )) then
13:      $FinishTime_{min}^* = FinishTime_{i,j}$ 
14:      $I_{min}^* = I_j$ 
15:   end if
16: end for
17: if ( $I_{min} \neq I_{min}^*$ ) then
18:   if (There is any unused instance with the same type as  $I_{min}$ ) then
19:     Remove the unused instance from  $Y$ 
20:   else
21:     if ( $I_{min}^* \neq Null$ ) then
22:       Assign  $t_i$  to  $I_{min}^*$ 
23:     end if
24:   end if
25: end if
26: if  $t_i$  is not assigned then
27:   Assign  $t_i$  to  $I_{min}$ 
28: end if
29: end for
30: return the generated schedule

```

The finish time t_i on all available resources is calculated in line 7. To calculate the finish time of t_i on I_j , we use Eqs. (2) and

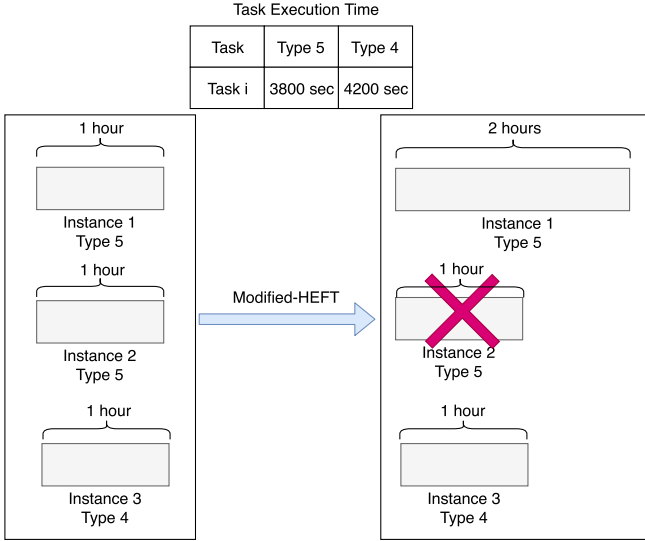


Fig. 4. An illustrative example showing the behavior of the modified-HEFT when the total execution time of the tasks hosted by an instance exceeds a full hour.

(3). When there is no other task already assigned to the resource I_j , the CPU available time is zero, in turn, the finish time of t_i is equal to its execution time; otherwise, all the available gaps between the already assigned tasks hosting by I_j are checked. The gaps starting before the completion of the predecessors of the task are filtered out. Then, if there is any gap bigger than the execution of t_i , t_i is placed within the first feasible gap. If there is enough space to execute the task, t_i is assigned at the end of the list of tasks of I_j . In line 8-11, we check for the minimum finish time. In line 12-15, we find the minimum finish time on the instance for which adding t_i does not change the cost.

In lines 17-28, we make a decision about the scheduling of the task with respect to the value of I_{min} and I_{min}^* . The case $I_{min} = I_{min}^*$ implies that assigning the task to the instance that yields the minimum finish time of the task does not increase the cost of running the instance. In this case, the task is easily assigned to I_{min} . However, when $I_{min} \neq I_{min}^*$, assigning the task to I_{min} could result in a budget violation since GRP already used all the budget to take as much as instance that it affords. Indeed, GRP assumes that each of the taken instances will run only for maximum one hour. Hence, running an instance for more than one hour can lead to a budget violation.

Modified-HEFT employs a *Budget-Violation Avoidance* (BVA) mechanism for dealing with such situations. The BVA mechanism of Modified-HEFT first searches to find an unused instance with the same type as I_{min} . If there is such an instance, it is removed from the taken instance set Y . Removing this instance keeps the algorithm in a safe-mode in terms of violation of the budget, because, although assigning the task to I_{min} increases the cost of running the instance (since it needs to run for one additional hour), another instance from the same type which exists in Y is removed to compensate this cost inflation. Fig. 4 shows an illustrative example for such a situation. This consideration enables the algorithm to run the tasks with an execution time longer than an hour, which is quite common for scientific workflow applications [10], without budget violation. In the example, as the earliest finish time of the $Task_i$

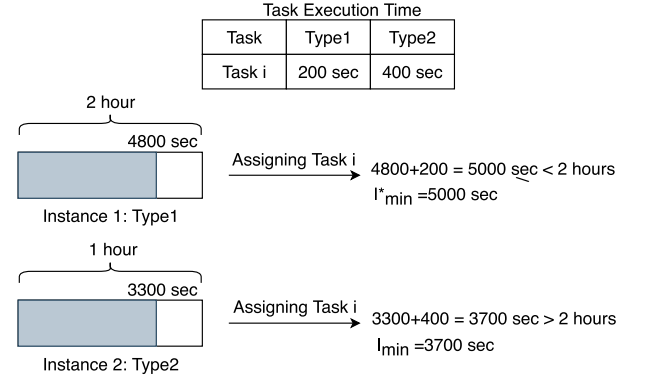


Fig. 5. An illustrative example showing the behavior of the modified-HEFT to prefer an instance with a longer finish time rather than the earliest finish time.

exceeds a full-hour and there is an unused instance with the same type as Instance 1, the unused instance 2 is removed, thereby the first instance can execute for one more hour. If there is no such an unused instance, the BVA mechanism of Modified-HEFT assigns the task to I_{min}^* , rather than I_{min} to avoid the risk of the budget violation.

Obviously, the finish time of the task on I_{min}^* may not be as good as on I_{min} , but this avoids the risk of a budget violation. Fig. 5 illustrates such a situation using an example. In this example, the earliest finish time of the i th task is 3,700 which is the case where the task is assigned to Instance 2. However, since assigning the task to Instance 2 increases the cost of Instance 2, the modified-HEFT assigns the task to the first instance. Here if we use HEFT, task i would be assigned to the second instance, since HEFT does not care about the budget and only focuses on minimizing the makespan. The worst-case that may happen is the case where $I_{min}^* = \infty$, meaning that there is no instance that can host the task without expanding the cost. In this case, the BVA mechanism has no other choice, hence, our Modified-HEFT keeps using I_{min} . After all tasks have been mapped, the workflow returns the generated schedule which includes the array X and Z in line 30.

In Fig. 6, we present an example to describe the fundamental idea of the proposed framework. This example shows the first iteration of GRP-HEFT when the budget is 2.2\$. After completing the first iteration illustrated in the example, the algorithm removes p_5 from InstanceTypeList and continues to choose from other instance types remaining in InstanceTypeList. Although in theory our algorithm may fail to find a feasible schedule for a given budget, in our experiments for all the cases that other methods can find a feasible schedule, our algorithm succeeds.

5 PERFORMANCE EVALUATION

In this section, we demonstrate the performance of GRP-HEFT through extensive evaluation considering four realistic scientific workflows including Montage, Cybershake, Sipt, and Inspiral [10] which are frequently used and studied in the context of scientific workflow scheduling [26], [27]. Furthermore, to assess the scalability of GRP-HEFT, we consider four different sizes for each workflow, 30, 50, 100, and 1,000 tasks. This is the same experimental setup used in state of the art work [26], [27]. These workflow applications are derived from

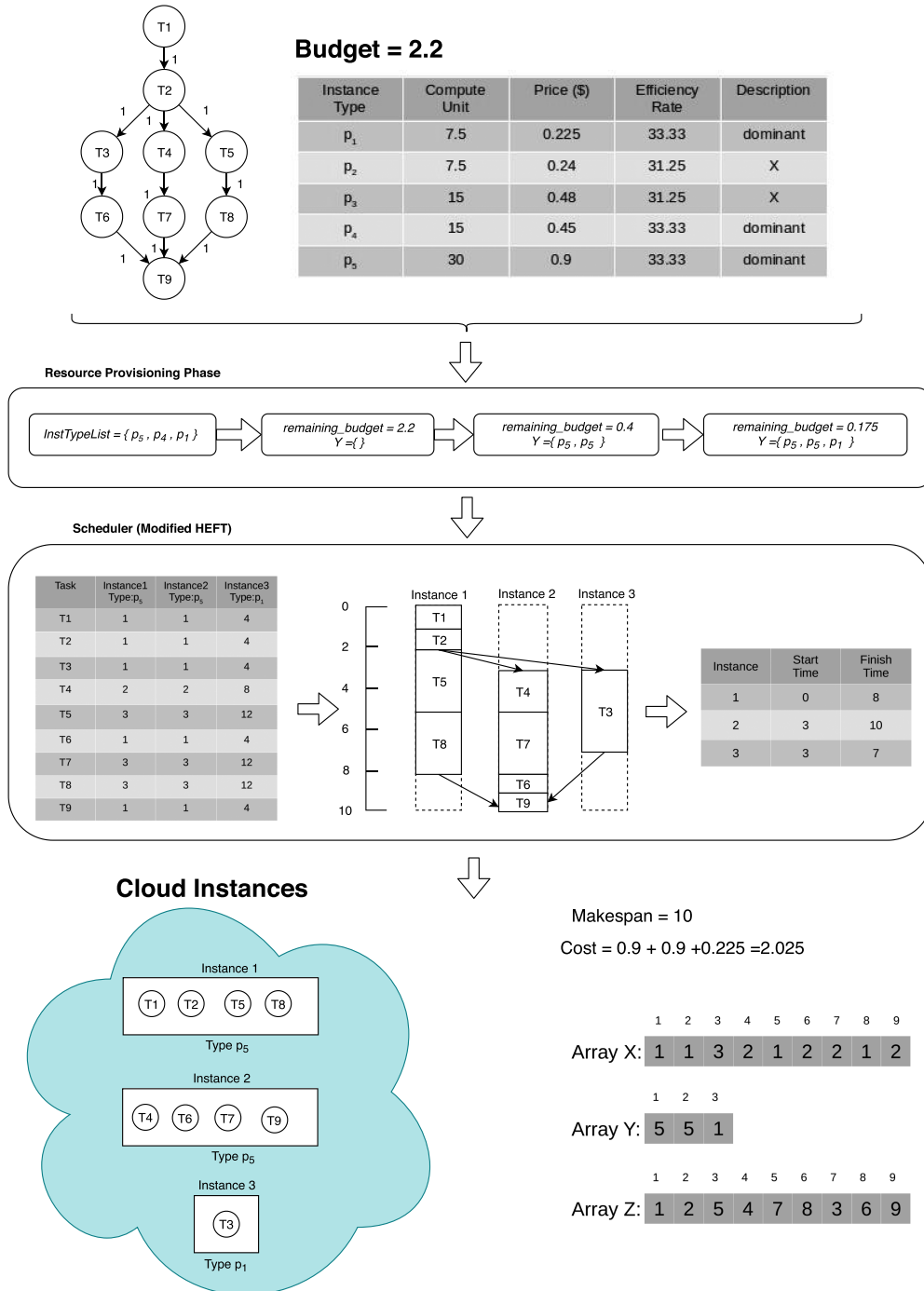


Fig. 6. Example of a solution obtained from the first iteration of the GRP-HEFT.

the Pegasus project [30] which publishes a number of real-world applications. The structure of these workflow applications is presented in Fig. 7 while their characteristics are listed in Table 1. All of these workflows are compute-intensive applications. Even for CyberShake that has the biggest size of the data compared to other workflows, given the average bandwidth of the EC2 cloud instances which is around 100 MB (see Table 2), the average execution time of tasks is multiple times bigger than the communication time. This is more relevant for other types of workflows that have a smaller size of data.

The experiments are conducted based on Amazon EC2 instances. The *General-Purpose* instance group in the USA

East region with the purchasing option *On-Demand* is used which is the same list of instances used in [2], [26], [27] to run their experiments. The characteristics of the instance types are provided in Table 2.

In order to evaluate GRP-HEFT, we develop an optimization framework where the proposed algorithms along with the state-of-the-art methods are all implemented. The optimization framework was coded in JAVA using the CloudSim toolkit [31]. Our optimization framework is available online in [32]. To generate the results, we run our framework on a server with Intel Xeon(R) 2.4 GHz CPUs with 80 cores and 128 GB RAM. This system runs under Linux CentOS 6.5 is running.

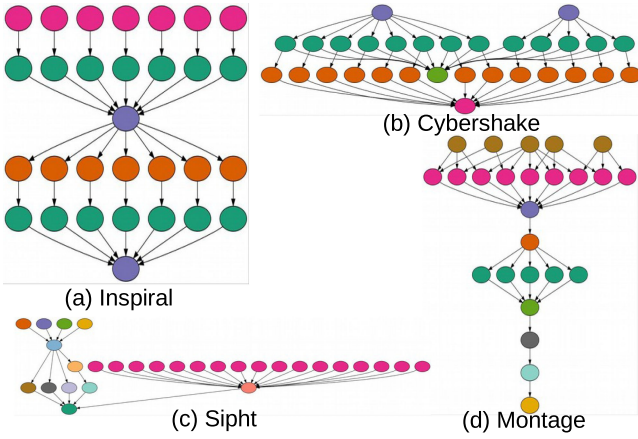


Fig. 7. Structure of four real-world workflow applications used in our experiments.

5.1 Baselines

As mentioned in Section 2.4, we compare GRP-HEFT with one multi-objective (cost versus makespan) optimization method which is MOACS, one single-objective optimization method based on the PSO algorithm which minimizes the makespan under a budget constraint, and one single-objective deadline-constrained workflow scheduling algorithm based on GA which minimizes the execution cost of workflows. It should be noted that all the parameters of the baseline algorithms are set based on the suggestions in their corresponding references. In the following, we explain how we conduct a comparison against each baseline.

- Baseline 1: The multi-objective optimization method, MOACS [27], generates a set of Pareto solutions, each of which is a pair of cost and makespan. To the best of our knowledge, there is no single quality indicator that can compare GRP-HEFT (a single objective

TABLE 1
Detailed Information of Real-World Workflow Applications Used in Our Experiments

DAG	Number of Nodes	Number of Edges	Average Data Size	Average Task Runtime (cu=1)
Montage 25	25	95	3.43 MB	8.44 s
Montage 50	50	206	3.36 MB	9.78 s
Montage 100	100	433	3.23 MB	10.58 s
Montage 1000	1000	4485	3.21 MB	11.36 s
CyberShake 30	30	112	747.48 MB	23.77 s
CyberShake 50	50	188	864.74 MB	29.32 s
CyberShake 100	100	380	849.6 MB	31.53 s
CyberShake 1000	1000	3988	102.29 MB	22.71 s
Sipht 30	30	91	7.73 MB	178.92 s
Sipht 60	60	198	6.95 MB	194.48 s
Sipht 100	100	335	6.27 MB	175.55 s
Sipht 1000	1000	3528	5.91 MB	179.05 s
Inspiral 30	30	95	9.00 MB	206.78
Inspiral 50	50	160	9.16 MB	226.19
Inspiral 100	100	319	8.93 MB	206.12 s
Inspiral 1000	1000	3246	8.90 MB	227.25 s

TABLE 2
Characteristics of Cloud Instances Used in Our Experiments

Instance Type Id	Instance Type Name	Compute Unit	Bandwidth (bytes/sec)	Price (\$)
p_1	m1.small	1.7	39,321,600	0.06
p_2	m1.medium	3.75	85,196,800	0.12
p_3	m3.medium	3.75	85,196,800	0.113
p_4	m1.large	7.5	85,196,800	0.24
p_5	m3.large	7.5	85,196,800	0.225
p_6	m1.xlarge	15	131,072,000	0.48
p_7	m3.xlarge	15	131,072,000	0.45
p_8	m3.2xlarge	30	131,072,000	0.9

optimization method) against multi-objectives. Hence, to compare against MOACS, for each Pareto solution, we run our algorithm once, such that we give the cost of the Pareto solution as the budget to GRP-HEFT, then compare the makespan of the Pareto solution with the result of our algorithm. For setting parameters, as suggested in [27], the population size is set to 10, both the pheromone local update rate and the pheromone global update rate are set 0.1, the termination condition of MOACS is to reach a maximum number of function evaluations which has been set to 60,000.

- Baseline 2: This is a straightforward comparison since PSO proposed in [6] includes the same objective, the same constraint, and the same cost model as what we use. For setting parameters of PSO, the number of particles is set to 100, while the number of iterations is 160. According to the experiments in [6], 100 particles and 160 iterations can result in the best makespan while the computation time of PSO is not extremely high.
- Baseline 3: To enable GA proposed in [23] to be compared with our budget-constrained workflow scheduling algorithm, we first run GA and its output which is the minimum cost of execution of the given workflow passes to GRP-HEFT as the budget constraint. Then we compare the output of GRP-HEFT with the deadline that was already given to GA. For setting parameters, we set population size to 100, the tournament selection method is used to create the next generation, the tournament size is set to 3, and the mutation rate 0.1 is used. In contrast to [23] where a time limit is defined as the termination condition, we limit the number of iterations of GA to $N/2$ where N is the number of tasks in the given workflow. The $N/2$ was achieved by applying the sensitivity analysis method proposed in [33] to reach the best performance within an acceptable computation time.

It is worth mentioning that since PSO, GA, and MOACS are not deterministic and may produce various results in different runs of the algorithm, we execute each of those algorithms for each specific workflow thirty times (95 percent confidence interval) and their average generated results are reported in our experiments. For example, for Inspiral 100, we run PSO 30 times and the average makespan of the 30 runs is considered as the makespan of PSO for Inspiral 100.

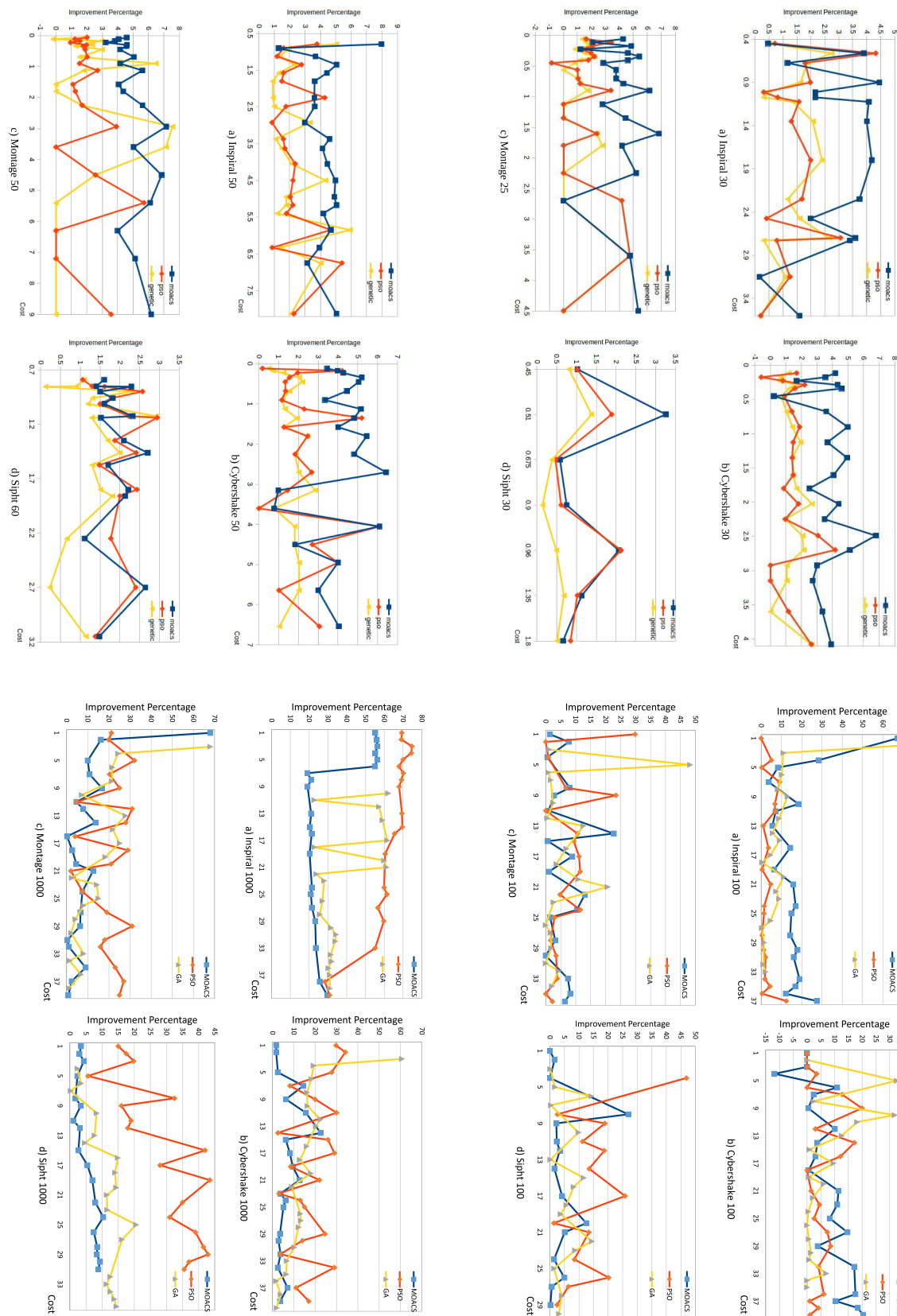


Fig. 8. Improvement of GRP-HEFT compared to baseline methods MOACS, PSO, and GA for all workflows.

5.2 Results

The experimental results for all workflows are presented in Fig. 8. This figure shows the percentage of improvement of GRP-HEFT against each of the baseline methods. To calculate

the improvement percentage for each baseline, we use $(1 - \frac{\text{Makespan}(\text{GRP-HEFT})}{\text{Makespan}(\text{baseline})}) \times 100$. When the value of a point in the chart is positive, indicates that our algorithm outperforms the baseline, and if it is negative, our algorithm achieves a

worse makespan than that of the considered baseline. For example, in Inspiral 100, GRP-HEFT improves the makespan, for cost = 1.463\$, by 67.16 percent in comparison with MOACS, while in CyberShake 100, for cost 3.45\$, GRP-HEFT achieves a worse makespan than MOACS by -11.8 percent.

More details about the experiments for all workflows are presented in Table 3. The first column of each table of this figure denotes the average improvement of GRP-HEFT against each of the other methods, in terms of the generated makespan. The second column shows that in what percentage of cases GRP-HEFT generates better results than the others. For example, for Inspiral 100, MOACS produced 19 Pareto solutions (i.e., 19 different budgets), and for no one of the budgets, we have got a worse result than MOACS. The third and fourth columns show similar information but for the same and the worse results. For example, in CyberShake 100, in 5.3 percent of the experiments, we got a worse result than MOACS. The two last columns reveal the best and the worst improvement of GRP-HEFT against the other methods. For example, in Inspiral 100, the best improvement of the makespan achieved by GRP-HEFT, among all budgets, against MOACS, PSO, and GA is 67.16, 12.41, and 68.89 percent, respectively. In other words, there is a Pareto solution generated by MOACS, for which our algorithm generates 67.16 percent better makespan. When the worst-performance is negative, means that there is at least one budget of a Pareto solution that GRP-HEFT generates a worse result than the considered alternative method.

Comparing the different size of workflows manifests that by growing the size of workflows the proposed algorithm more significantly outperforms other methods. The other indication revealed from Fig. 8 is that when the size of workflows is large (1,000 tasks) the second-best method (after GRP-HEFT) is MOACS. MOACS has stable performance and outperforms PSO and GA in all large-scale workflows, whereas for the smaller workflows including 30 and 50 tasks MOACS cannot surpass PSO and GA. For 30 tasks and 50 tasks workflows, GA and PSO have almost the same performance in terms of makespan while GA slightly outperforms. Furthermore, comparing GRP-HEFT with MOACS -as the second-best method for large-scale workflows- for large-scale workflows, the biggest improvement against MOACS is seen in Inspiral 1,000 (32.12 percent improvement) whereas the smallest improvement is seen in Sipht 1,000 (5.44 percent improvement).

As shown in Table 3, in all experiments for all type and size of workflows except for only a few experiments, we achieved significantly better results.

The reason to adopt the relative difference rather than absolute values in the diagrams is to make them more informative. By changing the budget value, the domain of the makespan of the workflows dramatically varies, and for the points with a lower value, the difference in the diagrams will not be tangible anymore. For example, for small budgets, the makespan of Inspiral 100 is in the scale of ten thousand seconds while for higher budgets this reduces to a few hundred seconds. Nevertheless, we also provide the absolute values of the makespan of Inspiral 100 and Inspiral 1,000 in Table 4. These tables also show the number of used instances taken by each algorithm as well as the average utilization of the taken instances. Due to space limitation, we

only listed the results of two workflows (Inspiral 100 and Inspiral 1,000). The detail of the results for other workflows is provided as an artifact in [32].

As the results manifest, for all workflows, irrespective of the size of data transferred between tasks and the average execution time of tasks, our algorithm outperforms other methods and this is not dependent on the compute-intensity of workflows. For example, for Sipht 1,000 and Inspiral 1,000 (which are the most compute-intensive workflows among those that we consider for our study) and CyberShake 100 (which uses the largest size of data) our algorithm obtains a noticeable improvement compared to other methods.

In terms of satisfiability, for all the workflows and all given budget values, GRP-HEFT has succeeded to find a feasible schedule. Looking at the tables in Table 4, for Inspiral 100 and Inspiral 1,000, all the algorithms can find feasible schedules for the given budgets except to PSO which fails to find a feasible schedule for Inspiral 100 when the budget is 1.463.

5.3 Complexity Analysis

To calculate the time complexity of our algorithm, we first need to calculate the time complexity of modified-HEFT which identical to the time complexity of HEFT. The time complexity of HEFT is $O(s \times N)$ [11] where N is the number of tasks and s is the number of computing resources. In our algorithm, the number of resources given to modified-HEFT is the number of taken instances in Y calculated in Line 8 to 12 of Algorithm 1. The maximum size of Y , is calculated as $\text{Min}\{\frac{\text{Budget}}{\text{Price}^{\text{cheapest}}}, N\}$ where $\text{Price}^{\text{cheapest}}$ is the price of the cheapest instance type. Its upper-bound is limited to N since the number of taken instances should not exceed the number of tasks. Hence, in the worst-case, the time complexity of modified-HEFT is $O(N^2)$. Accordingly, the time complexity of our algorithm is $O(mN^2)$ where m is the number of instance types. Nevertheless, since in practice $\frac{\text{Budget}}{\text{Price}^{\text{cheapest}}}$ is much less than N , the time complexity is close to $O(mN)$.

The time complexity of MOACS is $O(GRN^2s)$ where G is evolutionary generations, R is the colony size in the ant colony system, and s is the set of all instances in the cloud resource pool which is equal to $m \times N$. Therefore, its time complexity is at least $O(mN^3)$. This difference is clearly reflected in the comparison of the computation time of the algorithms (i.e., the time elapsed to execute the algorithm) listed in Table 5. The basic of GA algorithm proposed in [23] is the similar to EMS-C proposed in [26], hence having the same complexity order as EMS-C, i.e., $O(mN^3)$. The time complexity of PSO is $O(kgN^2)$ where k is the number of particles and g is the number of iterations. To have reasonable solutions in PSO, we set $k = 3 \times m$ and $g = N$ which yields $O(mN^3)$. Accordingly, GRP-HEFT has the lowest complexity compared to other baseline algorithms. The computation time of the algorithms is reported in Table 5.

6 DISCUSSION AND CONCLUSION

In this paper, we proposed a resource provisioning and a scheduling algorithm for budget-constrained workflow scheduling in cloud environments under an hourly-based cost model. As a resource provisioning mechanism, we proposed a greedy algorithm which lists the instance types according to

TABLE 3
Detailed Results Generated by the Three Baseline Methods in Comparison with GRP-HEFT for All the Workflows

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	2.73 %	15/15 = 100 %	0/15 = 0 %	0/15 = 0 %	4.45 %	0.16 %
PSO	1.47 %	15/15 = 100 %	0/15 = 0 %	0/15 = 0 %	4.32 %	0.2 %
GA	1.35 %	15/15 = 100 %	0/15 = 0 %	0/15 = 0 %	2.74 %	0.2 %

(a) Inspiral 30

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	4.11 %	19/20 = 95 %	1/20 = 5 %	0/20 = 0 %	6.83 %	0 %
PSO	1.61 %	14/20 = 70 %	5/20 = 25 %	1/20 = 5 %	4.76 %	-0.85 %
GA	1.30 %	14/20 = 70 %	6/20 = 30 %	0/20 = 0 %	4.76 %	0 %

(b) Montage 25

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	1.35 %	7/7 = 100 %	0/7 = 0 %	0/7 = 0 %	3.26 %	0.58 %
PSO	1.14 %	7/7 = 100 %	0/7 = 0 %	0/7 = 0 %	2.13 %	0.47 %
GA	0.62 %	7/7 = 100 %	0/7 = 0 %	0/7 = 0 %	1.38 %	0.14 %

(c) Sipt 30

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	3.77 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	6.78 %	0.21 %
PSO	1.46 %	17/20 = 85 %	2/20 = 10 %	1/20 = 5 %	4.17 %	-0.51 %
GA	1.40 %	19/20 = 95 %	1/20 = 5 %	0/20 = 0 %	2.63 %	0 %

(d) Cybershake 30

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	4.26 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	7.96 %	1.28 %
PSO	2.31 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	5.4 %	0.85 %
GA	2.26 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	5.02 %	0.9 %

(a) Inspiral 50

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	4.91 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	7.15 %	3.23 %
PSO	1.87 %	17/20 = 85 %	3/20 = 15 %	0/20 = 0 %	5.71 %	0 %
GA	2.10 %	13/20 = 65 %	6/20 = 30 %	1/20 = 5 %	7.55 %	-0.19 %

(b) Montage 50

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	1.88 %	16/16 = 100 %	0/16 = 0 %	0/16 = 0 %	2.7 %	1.11 %
PSO	1.92 %	16/16 = 100 %	0/16 = 0 %	0/16 = 0 %	2.94 %	1.3 %
GA	1.34 %	16/16 = 100 %	0/16 = 0 %	0/16 = 0 %	2.91 %	0.23 %

(c) Sipt 60

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	4.05 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	6.43 %	0.77 %
PSO	2.28 %	19/20 = 95 %	1/20 = 5 %	0/20 = 0 %	6.09 %	0 %
GA	1.62 %	20/20 = 100 %	0/20 = 0 %	0/20 = 0 %	2.86 %	0.51 %

(d) Cybershake 50

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	17.39%	18/18 = 100%	0/18 = 0%	0/18 = 0%	67.16%	3.79%
PSO	3.53%	18/18 = 100%	0/18 = 0%	0/18 = 0%	12.41%	0.31%
GA	9.83%	18/18 = 100%	0/18 = 0%	0/18 = 0%	68.89%	0.52%

(a) Inspiral 100

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	8.8%	18/18 = 100%	0/18=0%	0/18 = 0%	22.71%	0.52%
PSO	7.43%	18/18 = 100%	0/18 = 0%	0/18 = 0%	29.84%	0.87%
GA	6.74%	18/18 = 100%	0/18 = 0%	0/18 =0%	48.21%	0.48%

(b) Montage 100

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	4.78%	15/15 = 100%	0/15 = 0%	0/15 = 0%	26.91%	0.03%
PSO	14.59%	15/15 = 100%	0/15 = 0%	0/15 = 0%	46.91%	1.32%
GA	5.81%	15/15 = 100%	0/15 = 0%	0/15 = 0%	14.43%	0.1%

(c) Sipt 100

	Average Improvement	Better Result Rate	Same Result Rate	Worse Result Rate	Best Performance	Worst Performance
MOACS	8.03%	18/19 = 94.7%	0/19=0%	1/19=5.3%	17.14%	-11.8%
PSO	6.08%	19/19 =100%	0/19=0%	0/19=0%	19.95%	0.96%
GA	7.05%	19/19 =100%	0/19=0%	0/19=0%	32.57%	0.46%

(d) Cybershake 100

	Average Improvement	Better Result Rate	Same or Worse Result Rate	Best Performance	Worst Performance
MOACS	32.12 %	20/20 = 100 %	0/20 = 0 %	56.34 %	19.13 %
PSO	62.50 %	20/20 = 100 %	0/20 = 0 %	74.71 %	28.37 %
GA	38.33 %	20/20 = 100 %	0/20 = 0 %	62.1 %	30.27 %

(a) Inspiral 1000

	Average Improvement	Better Result Rate	Same or Worse Result Rate	Best Performance	Worst Performance
MOACS	10.08 %	20/20 = 100 %	0/20 = 0 %	68 %	0.2 %
PSO	17.92 %	20/20 = 100 %	0/20 = 0 %	31.97 %	3.01 %
GA	15.72 %	20/20 = 100 %	0/20 = 0 %	27.83 %	1.17 %

(b) Montage 1000

	Average Improvement	Better Result Rate	Same or Worse Result Rate	Best Performance	Worst Performance
MOACS	5.44 %	18/18 = 100 %	0/18 = 0 %	10.34 %	1.06 %
PSO	20.41 %	18/18 = 100 %	0/18 = 0 %	32.75 %	3.29 %
GA	10.79 %	18/18 = 100 %	0/18 = 0 %	20.58 %	0.24 %

(c) Sipt 1000

	Average Improvement	Better Result Rate	Same or Worse Result Rate	Best Performance	Worst Performance
MOACS	6.95 %	20/20 = 100 %	0/20 = 0 %	22.57 %	1.72 %
PSO	25.76 %	20/20 = 100 %	0/20 = 0 %	36.17 %	9.24 %
GA	14.90 %	20/20 = 100 %	0/20 = 0 %	60.69 %	1.65 %

(d) Cybershake 1000

TABLE 4

Absolute Values of Makespan, Budget, Number of Used Instances, and the Utilization of Instances for Inspiral 100 and Inspiral 1000

Inspiral 100

MOACS				PSO			GRP-HEFT			Genetic			
Cost/Budget	Makespan	No. Ins.	Util.	Makespan	No. Ins.	Util.	Makespan	No. Ins.	Util.	Deadline/makespan	No. Ins.	Util.	Cost
1.463	10612	5	0.9745	Infeasible	-	-	3485	3	1	10612	5	0.9140	1.575
2.025	3513	7	0.9814	2654	5	0.9997	2519	8	0.9949	3513	7	0.8603	1.688
2.25	2475	10	0.9161	2278	7	0.9858	2271	8	0.9909	2475	10	0.7963	2.431
2.7	1952	11	0.9639	2065	12	0.8793	1878	7	0.9985	1952	11	0.9325	2.987
3.375	1843	12	0.9639	1615	11	0.9416	1505	7	0.9996	1843	10	0.9205	3.263
3.6	1571	8	0.9950	1565	13	0.8896	1458	10	0.9807	1571	8	0.9296	3.713
4.05	1414	16	0.8661	1350	11	0.9613	1336	12	0.9450	1414	13	0.9050	4.276
4.5	1413	16	0.9273	1255	14	0.8643	1210	12	0.9629	1413	13	0.8590	4.223
4.98	1244	16	0.8944	1170	16	0.9055	1164	16	0.9231	1244	12	0.8243	4.733
5.85	1154	18	0.8338	1020	18	0.8921	972	16	0.8004	1154	11	0.8640	4.95
6.33	1078	16	0.9118	909	15	0.9241	895	14	0.9245	1078	13	0.9179	5.46
6.75	983	16	0.9305	849	17	0.8432	837	16	0.9086	983	13	0.9038	5.85
7.2	964	18	0.8522	830	14	0.8916	826	18	0.8424	964	12	0.9050	6.075
7.65	964	17	0.8672	801	17	0.9252	791	18	0.8039	964	13	0.9286	6.075
8.1	854	19	0.8790	733	16	0.8827	717	18	0.8815	854	13	0.8796	6.87
8.55	841	20	0.8663	695	17	0.8757	682	15	0.8721	841	13	0.8506	7.035
9	788	18	0.8831	685	19	0.8488	655	17	0.8569	788	12	0.9092	7.538
9.9	711	21	0.8672	628	18	0.8756	625	20	0.8644	711	14	0.8917	8.325
11.7	700	24	0.8231	580	24	0.8202	508	18	0.8596	700	14	0.8490	8.385

Inspiral 1000

MOACS				PSO			GRP-HEFT			Genetic			
Cost/Budget	Makespan	No. Ins.	Util.	Makespan	No. Ins.	Util.	Makespan	No. Ins.	Util.	Deadline/Makespan	No. Ins.	Util.	Cost
17.78	6890	48	0.9672	11767	54	0.9418	3088	48	0.9337	6890	48	0.8095	21.516
18.297	6882	50	0.9220	11593	47	0.9380	3026	57	0.9391	6882	50	0.8286	20.434
18.709	6817	48	0.9069	10067	38	0.9436	2976	45	0.9055	6817	48	0.7833	21.943
18.822	6618	47	0.9487	9800	49	0.9226	2964	54	0.9055	6618	47	0.8229	21.917
19.338	6524	49	0.9255	9499	54	0.9253	2850	49	0.9386	6524	49	0.8304	20.972
19.377	6325	47	0.8903	9444	43	0.9329	2844	48	0.9417	6325	47	0.8335	20.745
19.59	3485	50	0.8961	8930	53	0.8921	2820	50	0.8973	3485	50	0.8297	20.693
20.295	3484	48	0.8981	8859	64	0.8917	2749	71	0.8745	3484	48	0.8206	21.578
20.415	3382	62	0.8826	8622	53	0.8563	2735	68	0.8708	3382	62	0.7938	22.141
20.82	3337	61	0.8453	8618	48	0.8786	2635	55	0.8322	3337	61	0.7853	22.876
21.045	3283	55	0.8258	8534	70	0.8504	2614	53	0.8422	3283	55	0.8049	22.366
21.495	3273	59	0.8543	7460	67	0.7899	2574	55	0.8747	3273	59	0.8223	21.998
21.81	3203	62	0.8499	6513	65	0.8500	2553	55	0.8770	3203	62	0.7844	23.896
22.26	3200	66	0.8323	6461	70	0.8375	2513	61	0.8678	3200	66	0.8011	23.484
22.29	3174	63	0.8689	6271	64	0.8182	2510	52	0.8637	3174	63	0.7737	23.648
22.71	3145	64	0.8436	5909	56	0.8932	2469	69	0.8409	3145	64	0.8089	23.567
23.16	3098	62	0.8345	5689	63	0.8363	2378	61	0.8636	3098	62	0.7838	24.009
23.745	3056	60	0.8391	5211	55	0.8172	2335	68	0.8460	3056	60	0.8112	24.383
24.975	3024	63	0.7311	3141	63	0.7808	2250	86	0.7847	3024	63	0.7959	24.596
26.295	2999	67	0.7949	3051	71	0.7822	2109	72	0.7945	2999	67	0.8075	24.983

their efficiency rate. For our scheduler, we modified the HEFT algorithm, such that it takes into account a given budget in the scheduling process. We compared our algorithm with three different baseline algorithms that are considered to be state-of-the-art. According to the results for large-scale workflows with 1,000 tasks, in all experiments (i.e., 100 percent) we

strictly outperform these algorithms. The average makespan improvement against MOACS, PSO, and GA is 13.64, 31.64, and 19.93 percent, respectively. For workflows with 100 tasks in 99.32 percent of the experiments we outperform the baseline algorithms, and only in 0.68 percent we obtain a worse result than the best result generated by any of the baselines.

TABLE 5
Computation Times of the Algorithms for
Small-Scale and Large Scale Workflows

Workflow Size	GRP-HEFT	PSO	Genetic	MOACS
30 Tasks	35 ms	2100 ms	1600 ms	30 ms
50 Tasks	50 ms	5200 ms	2400 ms	70 ms
100 Tasks	80 ms	12500 ms	3000 ms	120 ms
1000 Tasks	150 ms	70000 ms	67000 ms	390 ms

For workflows with 100 tasks, the results generated by GRP-HEFT improves the average makespan against MOACS, PSO, and GA by 9.75, 7.9, and 7.35 percent, respectively. For 50 tasks and 30 tasks workflows, GRP-HEFT outperforms other algorithms in 90 and 88.75 percent of all experiments, respectively.

The time complexity of GRP-HEFT in the average-case is $O(mN)$ and in the worst-case is $O(mN^2)$, whereas the time complexity of other baseline algorithms discussed in this paper is at least $O(mN^3)$. Our experiments also demonstrated a considerable benefit in terms of the actual computation time of GRP-HEFT against the other baselines.

The other advantage of GRP-HEFT against the considered baselines is that GRP-HEFT is a deterministic algorithm, which means that for every run of GRP-HEFT with the same input, it will derive the same solution. For instance, if the makespan produced by GRP-HEFT for the Inspiral 100 workflow is 1,210 with a budget of 4.5\$, for all runs of GRP-HEFT with the same input we get again 1,210. However, all the baselines (i.e., MOACS, PSO, and GA) are meta-heuristics which can yield different solutions for some runs of the algorithm while their average behavior demonstrates good performance. For example, the PSO average makespan for Inspiral 100 is 1,250 seconds. However, in one out of thirty runs, it may achieve a dramatically worse makespan such as 1,600 seconds, whereas GRP-HEFT in all runs produces the same low makespan for the same input. This makes our approach more stable than MOACS, PSO and GA.

For future work, we intend to extend GRP-HEFT to leverage *Spot Instances* provided by Amazon EC2 which are cheaper but also less reliable than conventional *On-Demand* instances.

ACKNOWLEDGMENTS

The work presented in this article was supported by the research project funded by "The Austrian Research Promotion Agency (FFG)" under contract no 868018.

REFERENCES

- [1] J. D. Ullman, "NP-complete scheduling problems," *J. Comput. Syst. Sci.*, vol. 10, no. 3, pp. 384–393, 1975.
- [2] J. J. Durillo and R. Prodan, "Multi-objective workflow scheduling in amazon EC2," *Cluster Comput.*, vol. 17, no. 2, pp. 169–189, 2014.
- [3] H. Arabnejad and J. G. Barbosa, "A budget constrained scheduling algorithm for workflow applications," *J. Grid Comput.*, vol. 12, no. 4, pp. 665–679, 2014.
- [4] R. Sakellariou, H. Zhao, E. Tsiakkouri, and M. D. Dikaiakos, "Scheduling workflows with budget constraints," in *Integrated Research in GRID Computing*. Berlin, Germany: Springer, 2007, pp. 189–202.
- [5] X. Lin and C. Q. Wu, "On scientific workflow scheduling in clouds under budget constraint," in *Proc. 42nd Int. Conf. Parallel Process.*, 2013, pp. 90–99.
- [6] X. Wang, B. Cao, C. Hou, L. Xiong, and J. Fan, "Scheduling budget constrained cloud workflows with particle swarm optimization," in *Proc. IEEE Conf. Collaboration Internet Comput.*, 2015, pp. 219–226.
- [7] M. Malawski, G. Juve, E. Deelman, and J. Nabrzyski, "Algorithms for cost-and deadline-constrained provisioning for scientific workflow ensembles in IaaS clouds," *Future Gener. Comput. Syst.*, vol. 48, pp. 1–18, 2015.
- [8] E. Caron, F. Desprez, A. Muresan, and F. Suter, "Budget constrained resource allocation for non-deterministic workflows on an IaaS cloud," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2012, pp. 186–201.
- [9] C. Q. Wu and H. Cao, "Optimizing the performance of big data workflows in multi-cloud environments under budget constraint," in *Proc. IEEE Int. Conf. Services Comput.*, 2016, pp. 138–145.
- [10] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Gener. Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.
- [11] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [12] M. A. Rodriguez and R. Buyya, "A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments," *Concurrency Comput., Practice Experience*, vol. 29, no. 8, 2017, Art. no. e4041.
- [13] W.-N. Chen and J. Zhang, "An ant colony optimization approach to a grid workflow scheduling problem with various QoS requirements," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 39, no. 1, pp. 29–43, Jan. 2009.
- [14] F. Wu, Q. Wu, Y. Tan, R. Li, and W. Wang, "PCP-B2: Partial critical path budget balanced scheduling algorithms for scientific workflow applications," *Future Gener. Comput. Syst.*, vol. 60, pp. 22–34, 2016.
- [15] W. Chen, G. Xie, R. Li, Y. Bai, C. Fan, and K. Li, "Efficient task scheduling for budget constrained parallel applications on heterogeneous cloud computing systems," *Future Gener. Comput. Syst.*, vol. 74, pp. 1–11, 2017.
- [16] H. Arabnejad, J. G. Barbosa, and R. Prodan, "Low-time complexity budget-deadline constrained workflow scheduling on heterogeneous resources," *Future Gener. Comput. Syst.*, vol. 55, pp. 29–40, 2016.
- [17] X. Zuo, G. Zhang, and W. Tan, "Self-adaptive learning PSO-based deadline constrained task scheduling for hybrid IaaS cloud," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 2, pp. 564–573, Apr. 2014.
- [18] M. Pant, R. Thangaraj, and A. Abraham, "A new PSO algorithm with crossover operator for global optimization problems," in *Innovations in Hybrid Intelligent Systems*. Berlin, Germany: Springer, 2007, pp. 215–222.
- [19] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Apr.–Jun. 2014.
- [20] L. Liu, M. Zhang, R. Buyya, and Q. Fan, "Deadline-constrained coevolutionary genetic algorithm for scientific workflow scheduling in cloud computing," *Concurrency Comput., Practice Experience*, vol. 29, no. 5, 2017, Art. no. e3942.
- [21] A. Deldari, M. Naghibzadeh, and S. Abrishami, "CCA: A deadline-constrained workflow scheduling algorithm for multicore resources on the cloud," *J. Supercomput.*, vol. 73, no. 2, pp. 756–781, 2017.
- [22] Q. Wu, F. Ishikawa, Q. Zhu, Y. Xia, and J. Wen, "Deadline-constrained cost optimization approaches for workflow scheduling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3401–3412, Dec. 2017.
- [23] H. Y. Shishido, J. C. Estrella, C. F. M. Toledo, and M. S. Arantes, "Genetic-based algorithms applied to a workflow scheduling algorithm with security and deadline constraints in clouds," *Comput. Elect. Eng.*, vol. 69, pp. 378–394, 2018.
- [24] S. Singh and I. Chana, "A survey on resource scheduling in cloud computing: Issues and challenges," *J. Grid Comput.*, vol. 14, no. 2, pp. 217–264, 2016.
- [25] R. Garg and A. K. Singh, "Multi-objective workflow grid scheduling using ϵ -fuzzy dominance sort based discrete particle swarm optimization," *J. Supercomput.*, vol. 68, no. 2, pp. 709–732, 2014.
- [26] Z. Zhu, G. Zhang, M. Li, and X. Liu, "Evolutionary multi-objective workflow scheduling in cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 5, pp. 1344–1357, May 2016.
- [27] Z.-G. Chen et al., "Multiobjective cloud workflow scheduling: A multiple populations ant colony system approach," *IEEE Trans. Cybern.*, vol. 49, no. 8, pp. 2912–2926, Aug. 2019.

- [28] F. Wu, Q. Wu, and Y. Tan, "Workflow scheduling in cloud: A survey," *J. Supercomput.*, vol. 71, no. 9, pp. 3373–3418, 2015.
- [29] E.-G. Talbi, *Metaheuristics: From Design to Implementation*, vol. 74. Hoboken, NJ, USA: Wiley, 2009.
- [30] E. Deelman *et al.* "Pegasus: A workflow management system for science automation," *Future Gener. Comput. Syst.*, vol. 46, pp. 17–35, 2015.
- [31] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw., Practice Experience*, vol. 41, no. 1, pp. 23–50, 2011.
- [32] H. Faragardi and H. Modaberi, "Workflow scheduling on cloud instance," 2018. [Online]. Available: <https://github.com/workflow-scheduling-on-cloud-instances>
- [33] H. R. Faragardi, S. Dehnavi, T. Nolte, M. Kargahi, and T. Fahringer, "An energy-aware resource provisioning scheme for real-time applications in a cloud data center," *Softw., Practice Experience*, vol. 48, no. 10, pp. 1734–1757, 2018.



Hamid Reza Faragardi received the BSc degree in computer engineering and the MSc degree in software engineering from the University of Tehran, Tehran, Iran, in 2010 and 2012, respectively and the PhD degree in computer engineering from Malardalen University, Västerås, Sweden, in 2017. Since January 2018 to March 2019, he was a postdoc researcher with the University of Innsbruck. He was also a lecturer with the KTH Royal Institute of Technology in Stockholm where he supervised multiple master students in the context of artificial intelligence. He is currently a researcher with the Ericsson Research in Stockholm, focusing on mobile edge services. He has published more than 30 peer-reviewed papers in well-known academic journals and conferences. His main research interests include edge computing, cloud computing, IoT, and optimization problems.



Mohamad Reza Saleh Sedghpour received the MS degree in computer networking from the Iran University of Science and Technology, Tehran, Iran, in 2017. He is currently working toward the PhD degree in the Distributed Systems Research Group, Computer Science Department, Umeå University, Umeå, Sweden. His research interest includes distributed and autonomic systems. Currently, he is working on autonomic resource management in cloud computing systems.



Saber Fazliahmadi is working toward the PhD degree in information systems at the University of Putra, Malaysia. His research interests include cloud computing, industrial IoT, human-computer interaction, resource scheduling, and optimization. He is also working on web-based clinic software applications aiming at reducing medical errors.



Thomas Fahringer received the DrTechn degree from the Vienna University of Technology, Wien, Austria, in 1993. He is currently a professor of computer science with the University of Innsbruck. He is leading a research group in the area of distributed and parallel processing which develops the ASKALON system to support researchers worldwide in various fields of science and engineering to develop, analyse, optimize, and run parallel and distributed scientific applications. Furthermore, he leads a research team that created the Insieme

parallelizing and optimizing compiler for heterogeneous multicore parallel computers. He was involved in numerous national and international research projects including 15 EU funded projects. He has published five books, 40 journal and magazine articles and more than 200 reviewed conference papers including three best/distinguished IEEE/ACM papers. He is a member of the IEEE and ACM.



Nayereh Rasouli received the BSc degree in computer science from the University of Science and Culture, Tehran, Iran, and the MSc degree in computer science from the Qazvin Islamic Azad University, Qazvin, Iran, in 2013. Her research interests include cloud computing, communication networks, and IoT.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.